

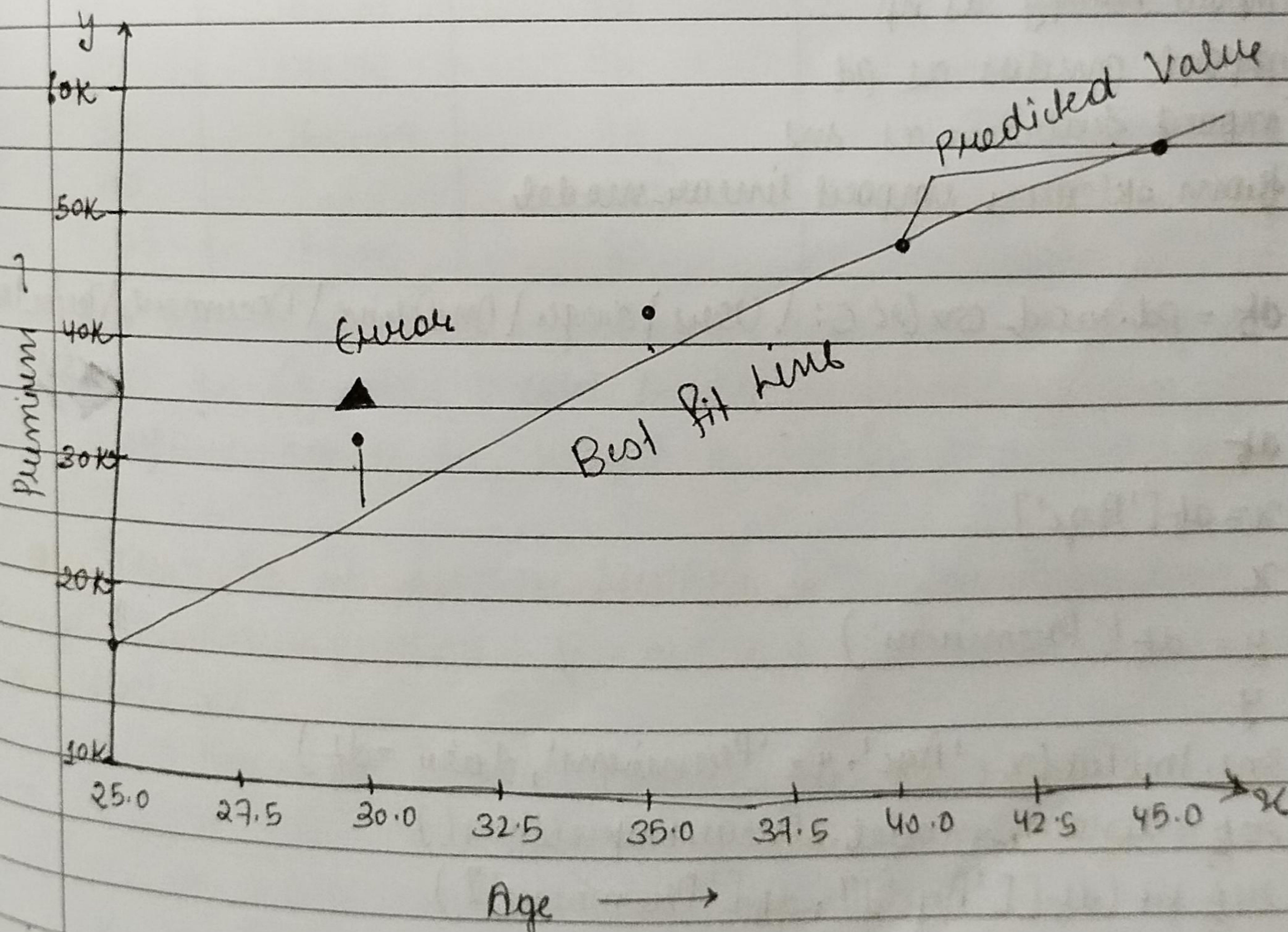
Expt No.:

★ Linear Regression

Linear Regression is a machine learning algorithm based on supervised learning.

It is a statistical method that is used for predictive analysis. Linear Regression makes predictions for continuous / Real or numeric variables such as cost, age, sales, temperature, product price, etc.

● Best-Fit line



Teacher's Signature _____

• Independent & Dependent Variable

$$y = mx + c \quad (\text{linear equation})$$

where,

$y \rightarrow$ dependent variable

$x \rightarrow$ Independent variable

$m \rightarrow$ slope / gradient / coefficient

$c \rightarrow$ Intercept

$$\text{Premium} = m * \text{age} + c$$

• Code (Term-Insurance.csv)

```
import numpy as np
```

```
import pandas as pd
```

```
import seaborn as sns
```

```
from sklearn import linear_model
```

```
df = pd.read_csv('C:\Users\Singh\OneDrive\Documents\Term-Insurance.csv')
```

```
df
```

```
x = df['Age']
```

```
x
```

```
y = df['Premium']
```

```
y
```

```
sns.heatmap(x='Age', y='Premium', data=df)
```

```
reg = linear_model.LinearRegression()
```

```
reg.fit(df[['Age']], df['Premium'])
```

```
reg.predict([[21]])
```

```
reg.coef_
```

```
// 1780.0
```

```
reg.intercept_
```

```
// -23500
```


Expt No.:

• verify linear Equation with code

let's $y = mx + c$

where, $m = 1780$, $x = 25$ and $c = -23500$

$$y = (1780) * (25) + (-23500)$$

$$y = 34800 - 23500$$

$$y = 11300 \quad (\text{verified})$$

★ Term Insurance Dataset

Age	height	weight	Premium
25	162.56	70	18000
30	172.72	95	38000
35	167.64	78	38000
40		110	60000
45	167.48	85	70000

We will find out Premiums whose age is 25 height is 165.56 and weight is 60

whose age is 60, height is 165.40 and weight is 80.

• Formula of linear Multiple Variable Regression.

Linear Equation: $y = m_1 * x_1 + m_2 * x_2 + m_3 * x_3 + c$

such as:

$y \rightarrow$ dependent variable (Premium)

$x_1 \rightarrow$ independent variable (Age)

$x_2 \rightarrow$ independent variable (Height)

$x_3 \rightarrow$ independent variable (Weight)

$c \rightarrow$ intercept

$m_1, m_2, m_3 \rightarrow$ slope / coefficient / gradient

Teacher's Signature _____

● Code

```
import pandas as pd
from sklearn import linear_model
```

```
df = pd.read_csv('C:\Users\single\OneDrive\Documents\Insurance.csv')
df
```

```
mean_height = df.Height.mean()
mean_height
```

```
df.Height = df.Height.fillna(mean_height)
df
```

```
reg = linear_model.LinearRegression()
reg.fit(df[['Age', 'Height', 'Weight']], df['Premium'])
```

```
reg.coef_           // array [[2150.26052416, -248.45851674,
reg.intercept_      // -16827.013154824956          312.65291961])
```

```
reg.predict([[27, 165.56, 60]])           // array (18854.40430838)
reg.predict([[60, 165.10, 80]])           // array (96180.35091498)
```


Verify with Linear Equations

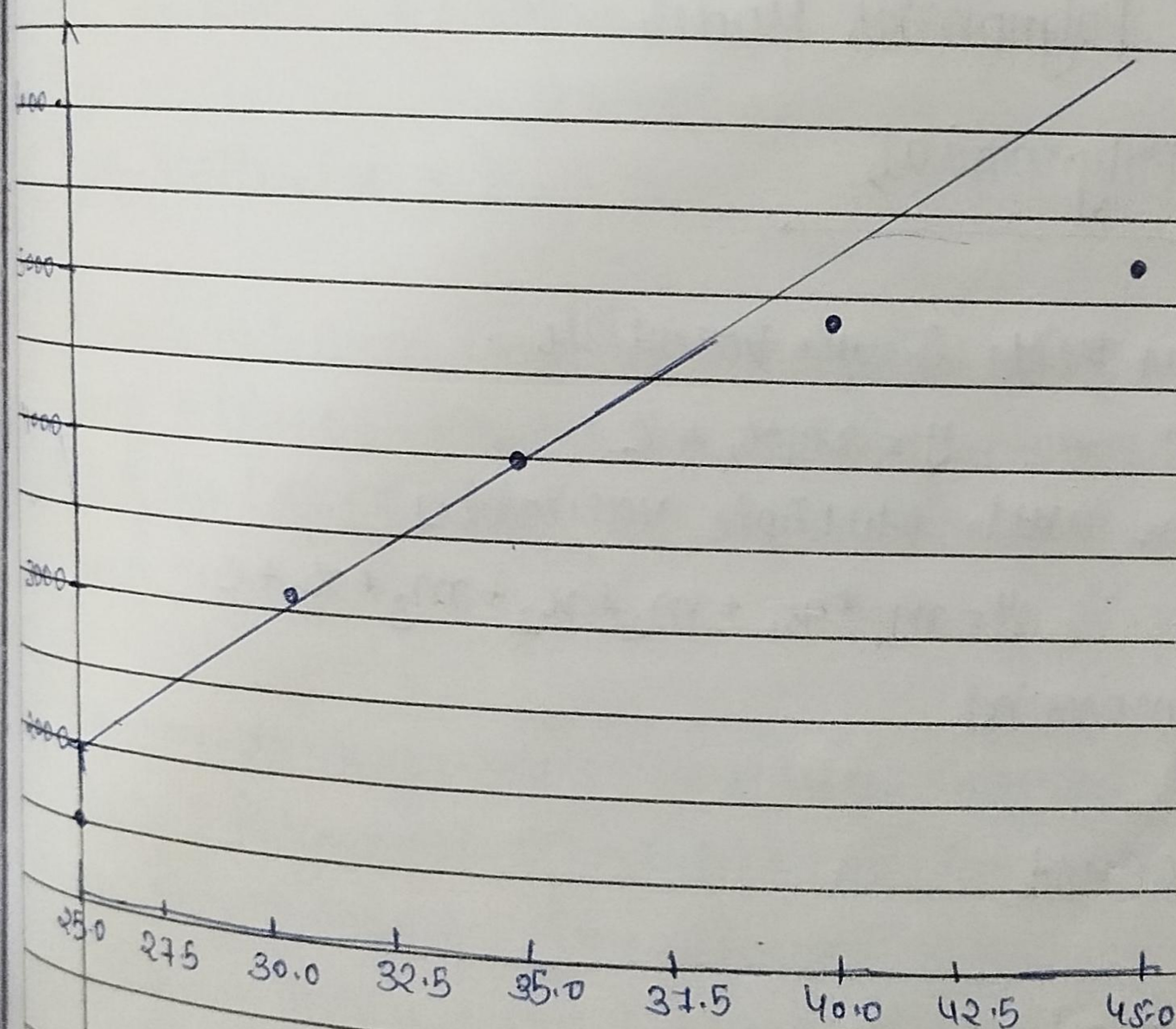
$$\text{let's } y = m_1x_1 + m_2x_2 + m_3x_3 + c$$

$$y = (2150.26052416)(27) + (-248.45851374)(165.56) + (312.65291961)(60) + (-16827.013154824956)$$

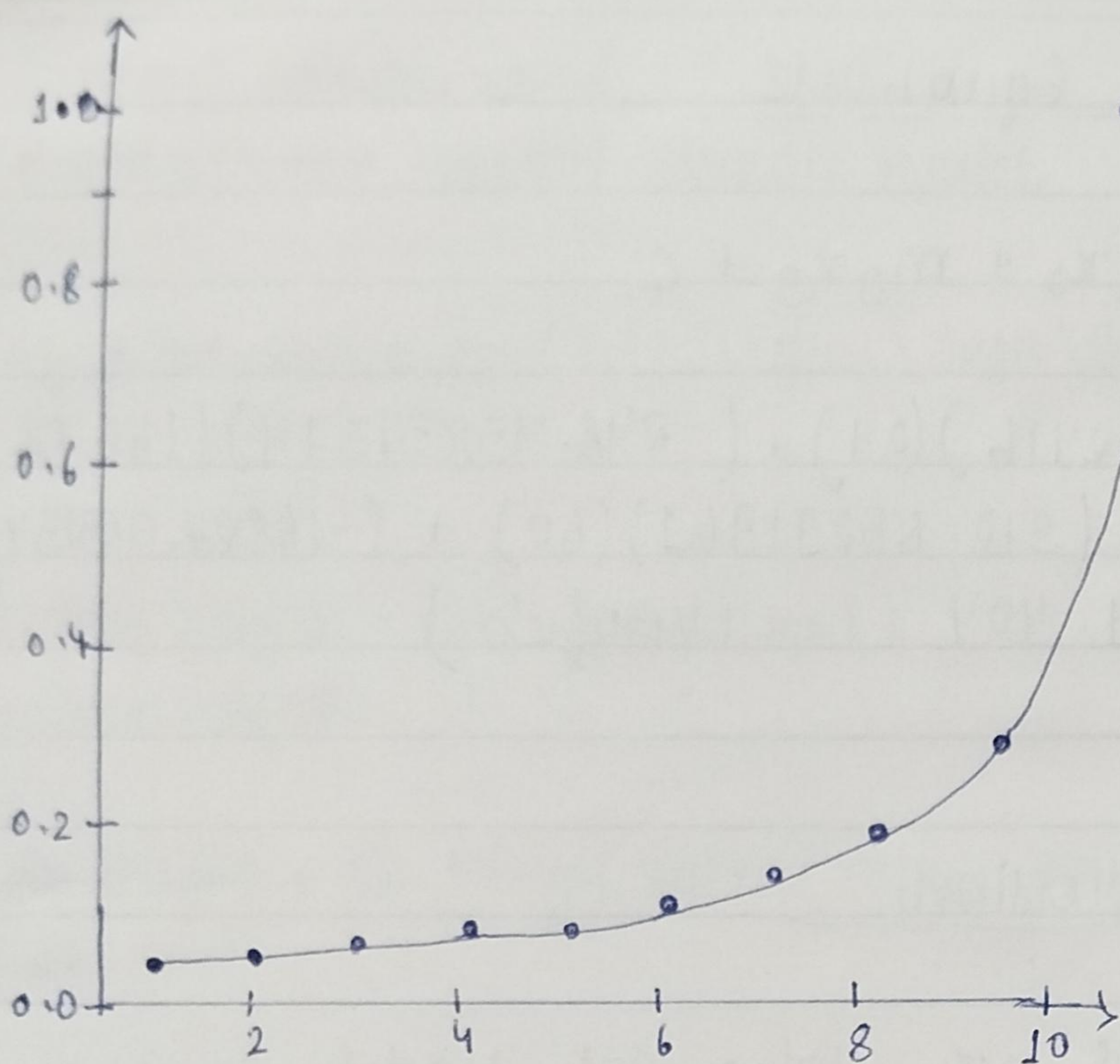
$$y = 18854.404 \quad (\text{verified})$$

★ Polynomial Regression

Simple Linear Model vs Polynomial Model graphs



Simple Linear Model



Polynomial Model

★ Degree of Polynomial

Linear Equation with Single Variable

$$y = m_1 x_1 + c$$

Linear Equation with Multiple Variables

$$y = m_1 * x_1 + m_2 * x_2 + m_3 * x_3 + c$$

Degree of Polynomial

0 Degree

$$y = \text{const}$$

1 Degree

$$y = m x + c$$

2 Degree

$$y = a x^2 + b x + c$$

Generalized Polynomial

$$y = a_0 + a_1 x + a_2 x^2 + a_3 x^3 + \dots + a_n x^n$$

Expt No.:

• Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv(r"C:\Users\singh\OneDrive\Documents\Position-Salaries.csv")
```

```
df
x = df.iloc[:, 1:2].values
```

```
x
```

```
y = df.iloc[:, 2].values
```

```
y
```

```
plt.scatter(x, y)
```

```
from sklearn import linear_model
```

```
reg = linear_model.LinearRegression()
```

```
reg.fit(df[['level']], df[['Salary']])
```

```
reg.predict([[6.5]]) // 330378.7878
```

```
from sklearn.preprocessing import PolynomialFeatures
```

```
poly = PolynomialFeatures(degree = 2)
```

```
x_poly = poly.fit_transform(x)
```

```
reg2 = linear_model.LinearRegression()
```

```
reg2.fit(x_poly, y)
```

```
reg2.predict(poly.fit_transform([[6.5]])) // 189498.106
```


★ Logistic Regression (Binary classification)

Logistic Regression is a machine learning algorithm based on supervised learning.

It is a statistical method that is used for predicting probability of target variables.

Logistic Regression makes probability for classification problems that are discrete in nature.

- ✓ Binary Classification
- ✓ Multiclass classification

Ex: win or loss, dead or alive

Ex: onion or potato or Tomato

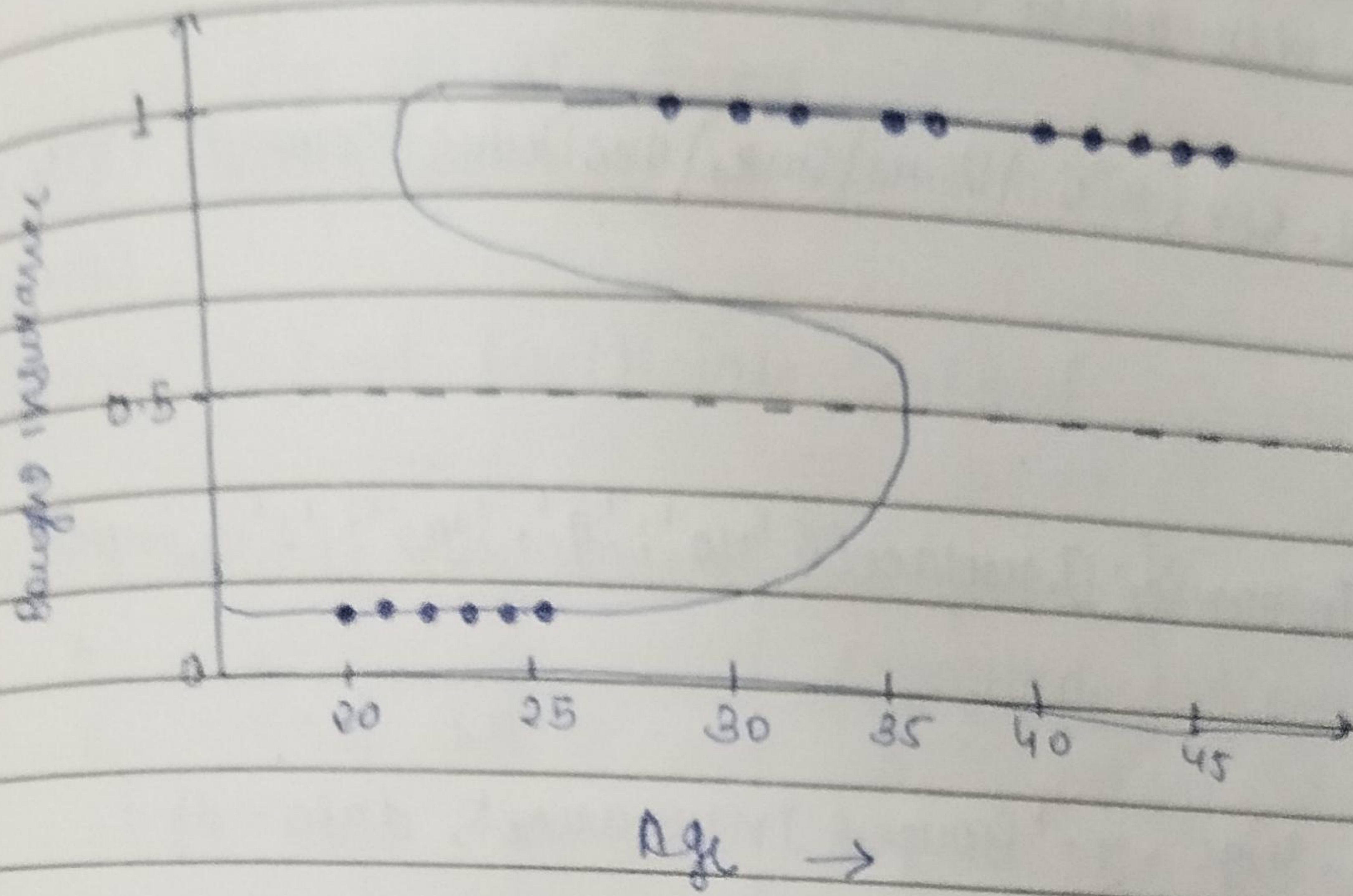
● Logistic Regression Real Life Example

- > Heart Attack prediction
- > Tumour prediction
- > Credit-Card Fraud detection
- > Spam detection

● Tins Insurance Dataset

Age	Bought Insurance
21	no
48	yes
32	yes
41	yes
20	no
35	yes
20	no
23	no

Bought Insurance
0
1
1
1
0
1
0
0

ChapterLogistic Regression Graph

Formula of Sigmoid Function will be

$$y = 1 / 1 + e^{-x}$$

converts input (x) into range 0 to 1.

such as,

$y \rightarrow$ dependent variable (Bought insurance)

$x \rightarrow$ independent variable (Age)

$e \rightarrow$ Euler's Constant ≈ 2.71828

• Code

```
import pandas as pd
import matplotlib.pyplot as plt
```

```
df = pd.read_csv("C:\\Users\\Singh\\OneDrive\\Documents\\TERM-INSURANCE-  
- DATASET.csv")
```

```
df
```

```
df['Bought Insurance'].replace({'no': '0', 'yes': '1'}, inplace=True)
```

```
df
```

```
plt.scatter(x='Age', y='Bought Insurance', data=df)
```

```
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(df[['Age']],
                                                    df['Bought Insurance'], test_size=0.2)
```

```
x_train
```

```
len(x_train)
```

```
x_test
```

```
len(x_test)
```

```
from sklearn.linear_model import LogisticRegression
```

```
m = LogisticRegression()
```

```
m.fit(x_train, y_train)
```

```
m.predict(x_test)
```


- * Multiclass classification Real Examples
 identifying animals types from photos
 identifying vehicles types
 news categorization

Sample Student Multiclass Dataset

Hours Studied	Attendance	Assignment Submitted	Previous Score	Extra-curricular	Final Result
1	65	3	45	No	Low
2	70	4	50	No	Med
3	75	5	55	Yes	Med
4	80	6	60	Yes	Med
5	85	7	65	Yes	High
6	90	8	70	Yes	High
7	92	9	75	Yes	High

Code

```
import pandas as pd
```

```
df = pd.read_csv(r"C:\Users\Singer\Downloads\Student-Multiclass-dataset.csv")
```

```
df['Extra-curricular'].replace({'No': '0', 'Yes': '1'}, inplace=True)
```

```
df['Final-Result'].replace({'Low': '1', 'Medium': '2', 'High': '3'}, inplace=True)
```

```
df
```


from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(df[['Hours-Studied',
'Attendance', 'Assignments-Submitted', 'Previous-Score',
'Extracurricular']], df['Final-Result'],
test_size=0.2)

x_train
len(x_train)
y_test
len(y_test)

from sklearn.linear_model import LogisticRegression

lr = LogisticRegression()
lr.fit(x_train, y_train)
lr.predict(x_test)
lr.score(x_test, y_test)

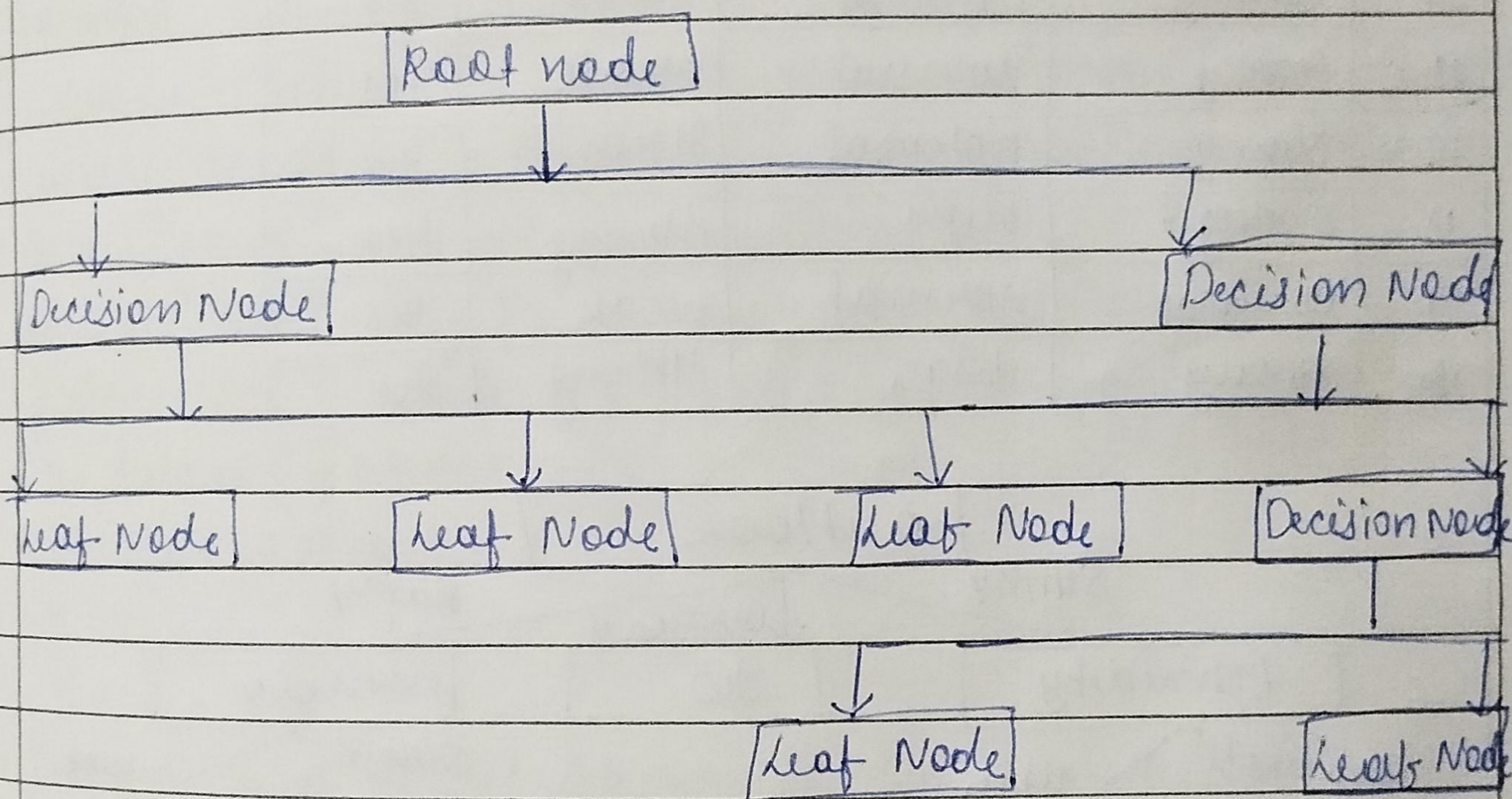
import seaborn as sns

sns.pairplot(df[['Hours-Studied', 'Attendance', 'Assignments-Submitted', 'Previous-Score', 'Extracurricular',
'Final-Result']], hue='Final-Result')

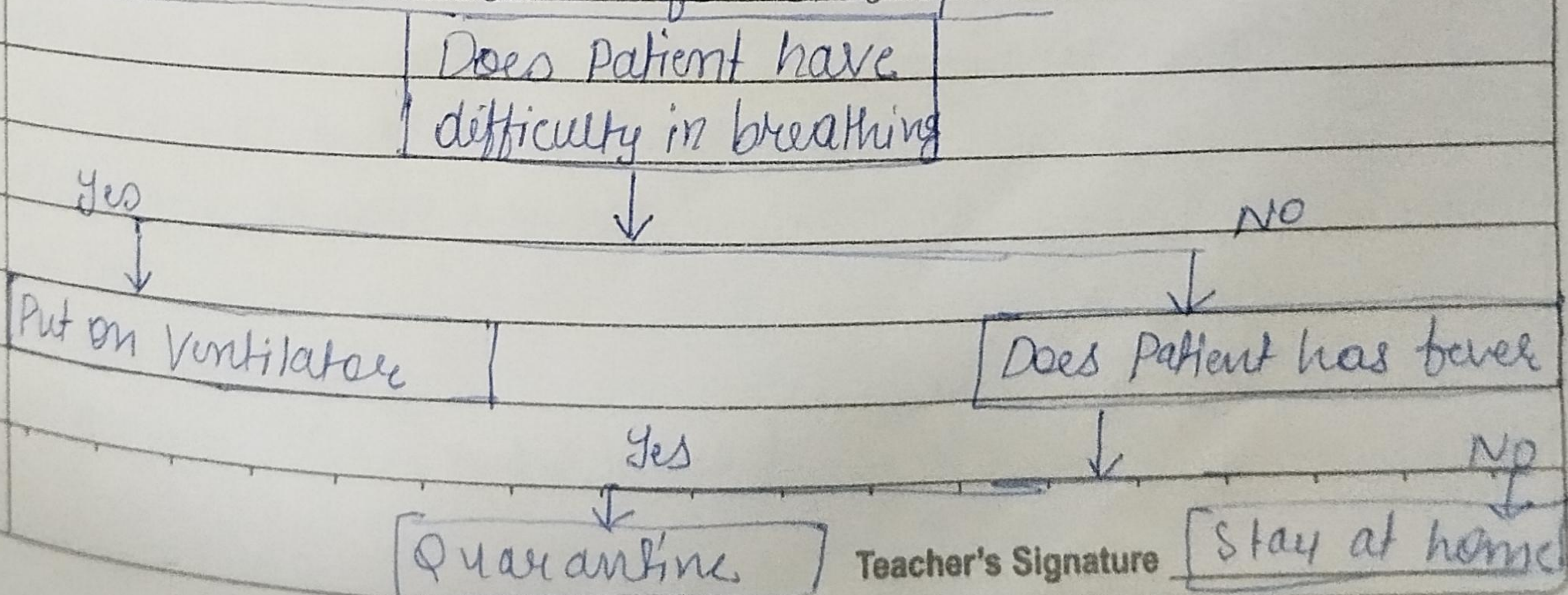
★ Decision Tree in ML

Decision Tree is a machine learning algorithm based on supervised learning, that can be used for both regression and classification problems.

The goal is to build up a classifier model that can predict the class or value of the target variable.



● Decision Tree Real Life Example

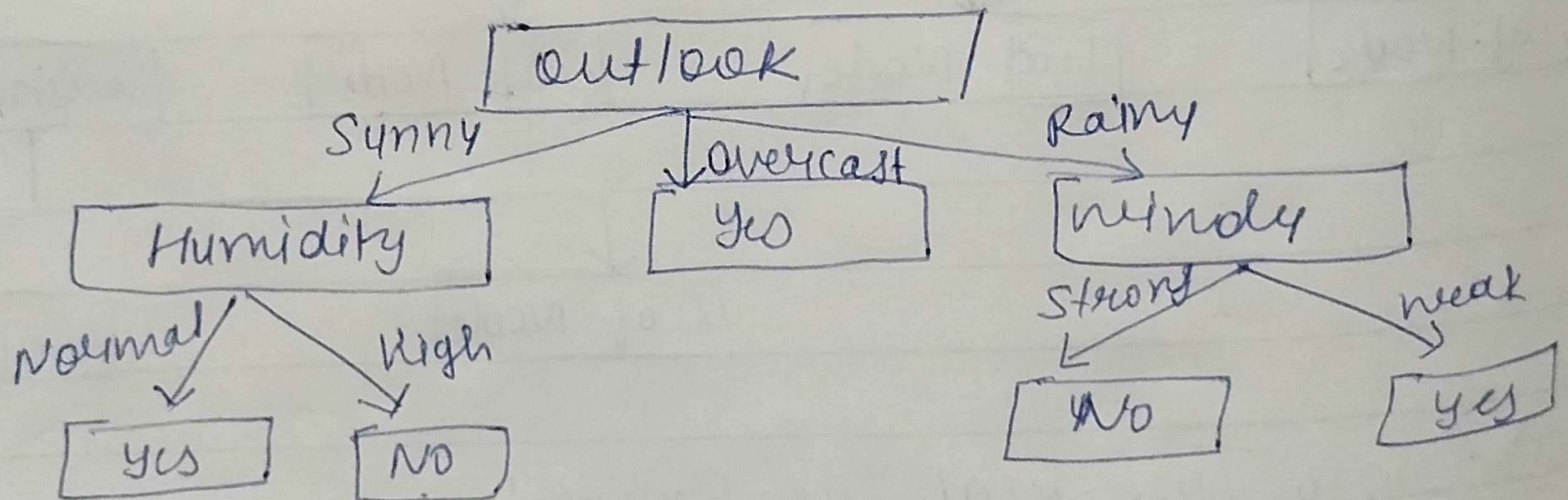


Teacher's Signature

Medical Field

● How we create a Decision Tree?

	outlook	humidity	wind	Play	
1	Sunny	high	weak	no	Sunny (2)
2	Sunny	high	strong	no	Overcast (0)
3	Sunny	high	weak	yes	Rainy (1)
4	overcast	high	weak	yes	no (0)
5	rainy	high	weak	yes	yes (1)
6	rainy	normal	weak	no	weak (1)
7	rainy	normal	strong	yes	strong (0)
8	overcast	normal	weak	no	high (0)
9	Sunny	high	weak	yes	normal (1)
10	Sunny	normal	weak	yes	
11	rainy	normal	strong	yes	
12	Sunny	normal	strong	yes	
13	overcast	high	strong	yes	
14	overcast	normal	weak	yes	
15	rainy	high	strong	no	




```

Code
import pandas as pd
dataset = pd.read_csv("C:\\Users\\Singh\\Downloads\\
decision-tree-dataset.csv")

dataset
from sklearn.preprocessing import LabelEncoder
outlook = LabelEncoder()
Humidity = LabelEncoder()
Wind = LabelEncoder()
Play = LabelEncoder()
dataset['Outlook'] = outlook.fit_transform(dataset['Outlook'])
dataset['Humidity'] = Humidity.fit_transform(dataset['Humidity'])
dataset['Wind'] = Wind.fit_transform(dataset['Wind'])
dataset['Play'] = Play.fit_transform(dataset['Play'])
features_col = ['Outlook', 'Humidity', 'Wind']
X = dataset[features_col]
y = dataset.Play

X
y
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
from sklearn.tree import DecisionTreeClassifier
classifier = DecisionTreeClassifier(criterion='gini')
classifier.fit(X_train, y_train)
classifier.predict(X_test)
X_test
y_test
classifier.score(X_test, y_test)

```

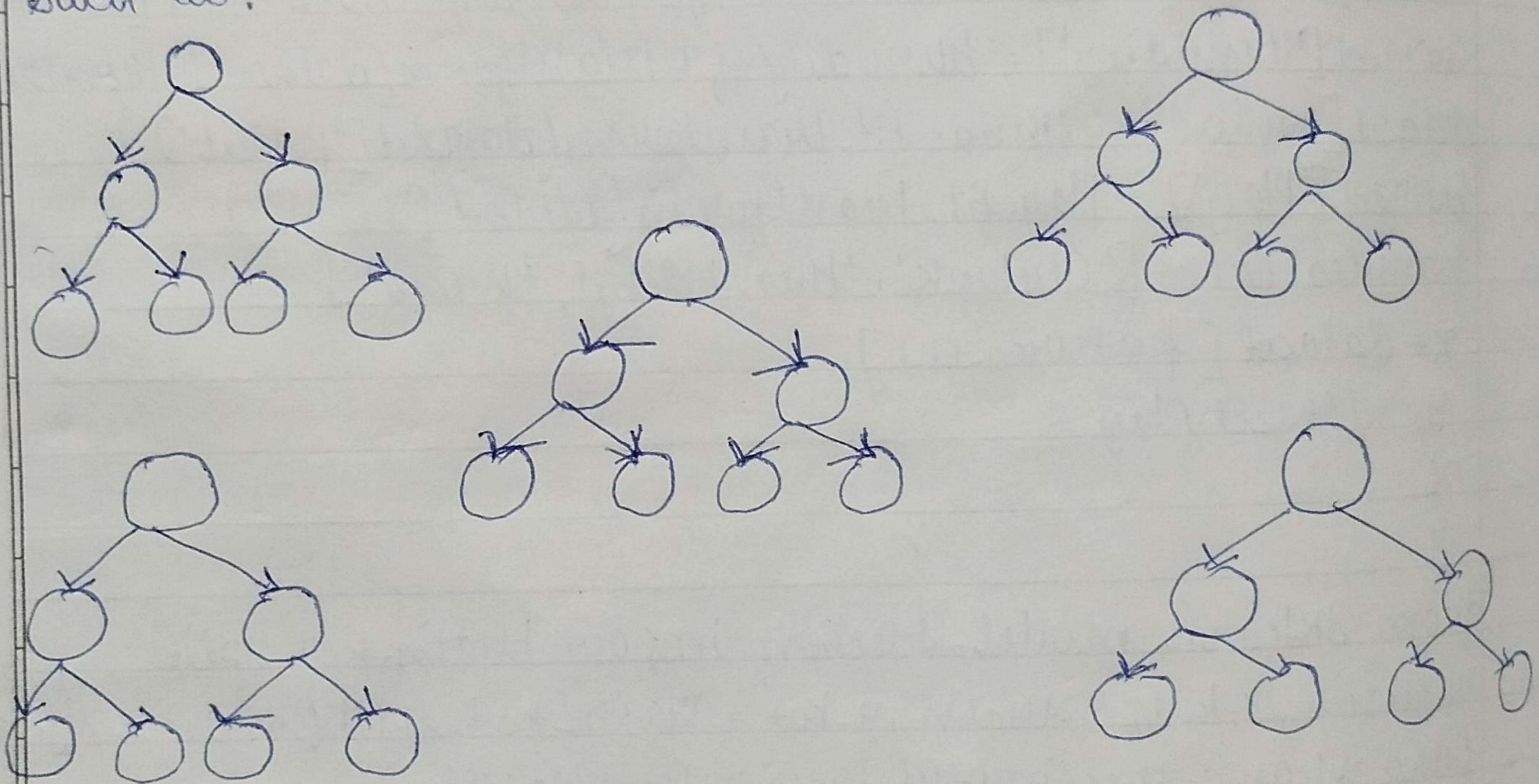

from sklearn import tree
tree.plot_tree(classifier)

★ Random Forest Algorithm

Random Forest is a machine learning algorithm based on supervised learning that can be used for both regression and classification problems.

It is a collection of multiple random decision trees, which is called the forest.

such as:

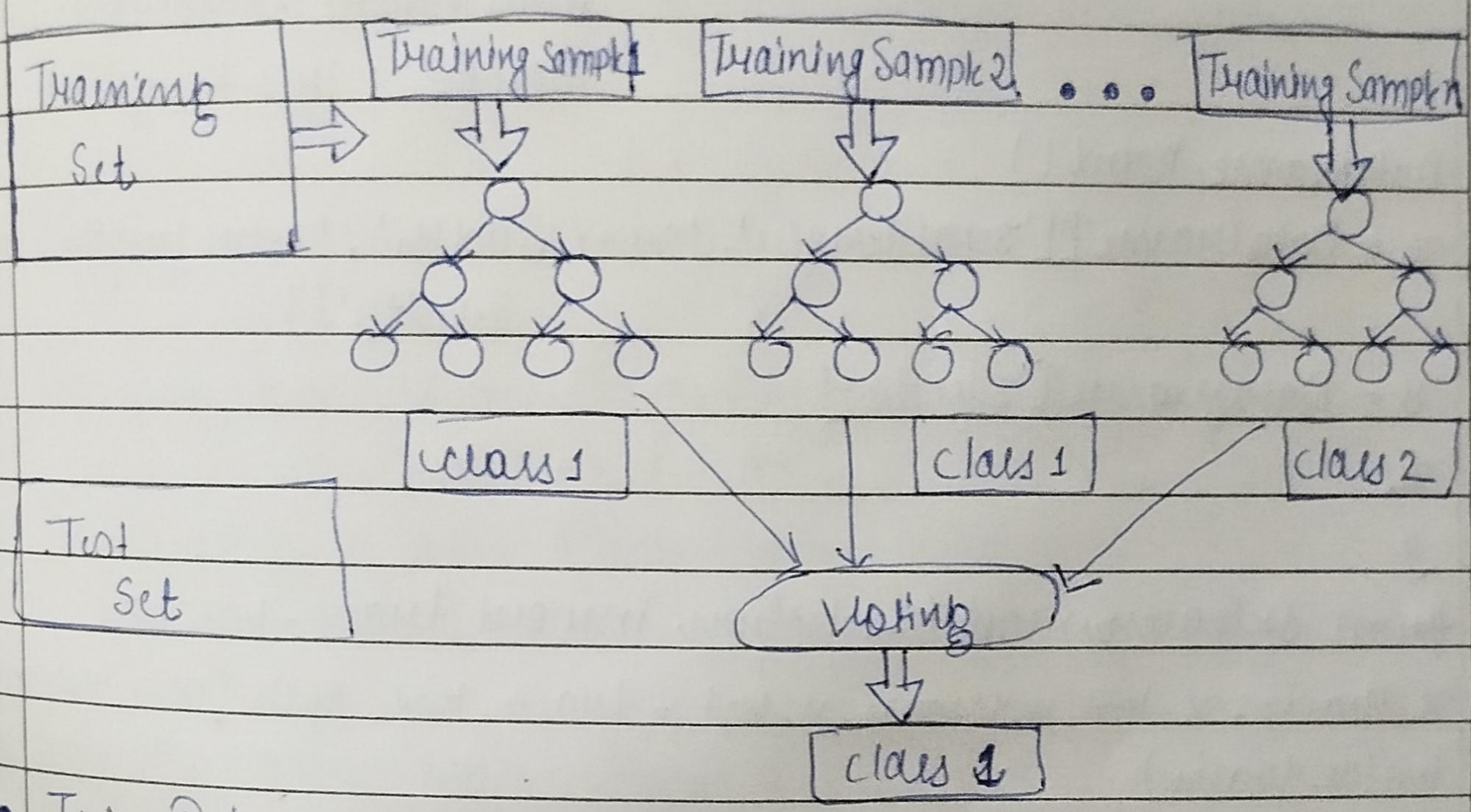


Random Forest Algorithm

Real life Example of RFA

- Financial Fraud detection
- Healthcare Diagnosis
- E-commerce Recommendations

How does the Algorithm works ?



Trip Dataset

SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
6.8	3.2	5.9	2.3	(2) Iris-Virginica
6.9	3.1	5.1	2.3	"
4.9	3.0	1.4	0.2	(0) Iris-Setosa
5.6	3.0	4.5	1.5	(1) Iris-Versicol
4.8	3.1	1.6	0.2	Iris-Setosa
5.8	2.8	5.1	2.4	Iris-Virginica
7.2	3.6	6.1	2.5	"
5.1	3.5	1.4	0.3	Iris-Setosa
4.7	3.2	1.6	0.2	"
6.6	3.0	4.4	1.4	Iris-Versicol

Teacher's Signature

• Code

```
from sklearn import datasets
```

```
iris = datasets.load_iris()
```

```
Dataframe.head()
```

```
iris
```

```
import pandas as pd
```

```
Dataframe = pd.readDataFrame({ 'sepal length': iris.data[:,0],  
                                'sepal width': iris.data[:,1],  
                                'petal length': iris.data[:,2],  
                                'petal width': iris.data[:,3],  
                                'species': iris.target })
```

```
Dataframe.head()
```

```
x = Dataframe[['sepal length', 'sepal width', 'petal length', 'petal  
width']]
```

```
y = Dataframe['species']
```

```
x
```

```
y
```

```
from sklearn.model_selection import train-test-split
```

```
x_train, x_test, y_train, y_test = train-test-split(x, y, test_size=0.3)
```

```
len(x_train)
```

```
len(x_test)
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
clf = RandomForestClassifier(n_estimators=100, criterion='gini')
```

```
clf.fit(x_train, y_train)
```

```
clf.predict(x_test)
```

```
x_test
```

```
clf.score(x_test, y_test)
```

```
clf.predict([[3.7, 2.6]])
```

```
feature_imp = pd.Series(clf.feature_importances_, index=iris.  
feature_names).sort_values(ascending=False)
```

feature_imp

* Naive Bayes Classifier Algorithm

Naive Bayes classifier is a machine learning algorithm based on supervised learning, that can be used for solving classification problems.

It is probabilistic classifier, which means it predicts on the basis of the probability of an object.

Such as:

> Tossing a coin Probability

$$S = \{H, T\}$$

> Tossing two coins Probability

$$S = \{HH, HT, TH, TT\}$$

> Throwing a dice Probability

$$S = \{1, 2, 3, 4, 5, 6\}$$

• Why the Name Naive Bayes?

> It comprises of two words Naive and Bayes:

Naive: It's called naive because it makes the assumption that all attributes are independent of each other.

such as: Fruit = {green, oval, sweet} $\Rightarrow$ watermelon

Bayes: It's called bayes because it depends on the Principle of Bayes Theorem.

What is Conditional Probability?

such as: Playing card example

4 Suits = Hearts, Clubs, Diamonds, Spades

13 cards = Ace, 2, 3, 4, 5, 6, 7, 8, 9, 10, Jack, Queen, King

Each of 13 cards has all 4 suits.

If you pick a card from the deck, can you guess the probability of getting a king given the card is a spade?

Total Spade = 13

King = 1

$$P(\text{King} / \text{Spade}) = 1/13 = 0.076$$

$P(A|B)$ = Probability of A occurring given that B has already occurred.

• What is Bayes Theorem?

The Naive Bayes classifier works on the principle of conditional probability, as given by the Bayes Theorem:

$$P(A|B) = \frac{P(B|A) * P(A)}{P(B)}$$

Where,

A, B = Events

$P(A|B)$ = Probability of A given that B is true.

$P(B|A)$ = Probability of B given that A is True.

$P(A)$ = Probability of event A.

$P(B)$ = Probability of event B.

Ex. No.:
 Examples of Bayes Theorem.

Let's take Spade & King card example:

$$P(A|B) = \frac{P(B|A) * P(A)}{P(B)}$$

$$P(\text{King}|\text{Spade}) = \frac{P(\text{Spade}|\text{King}) * P(\text{King})}{P(\text{Spade})}$$

given: $P(\text{Spade}|\text{King}) = \frac{1}{4}$, $P(\text{King}) = \frac{1}{13}$
 $P(\text{Spade}) = \frac{1}{4}$

$$P(\text{King}|\text{Spade}) \Rightarrow \frac{\left(\frac{1}{4}\right) * \left(\frac{1}{13}\right)}{\left(\frac{1}{4}\right)} \Rightarrow \frac{1}{52} \times \frac{4}{1} \Rightarrow \frac{1}{13}$$

$\Rightarrow 0.076$

• Where Naive Bayes is Used?

- Face Recognition
- Weather Forecast
- News Categorization

• Code

import pandas as pd

dataset = pd.read_csv(r"C:\Users\singh\Downloads\Social-Networks-Ads.csv")

dataset

x = dataset.iloc[:, [1, 2, 3]].values

x

y = dataset.iloc[:, -1].values

y


```
from sklearn.preprocessing import LabelEncoder
```

```
lb = LabelEncoder()
```

```
x[:,0] = lb.fit_transform(x[:,0])
```

```
x
```

```
from sklearn.model_selection import train_test_split
```

```
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2)
```

```
len(x_train)
```

```
len(x_test)
```

```
from sklearn.naive_bayes import GaussianNB
```

```
model = GaussianNB()
```

```
model.fit(x_train, y_train)
```

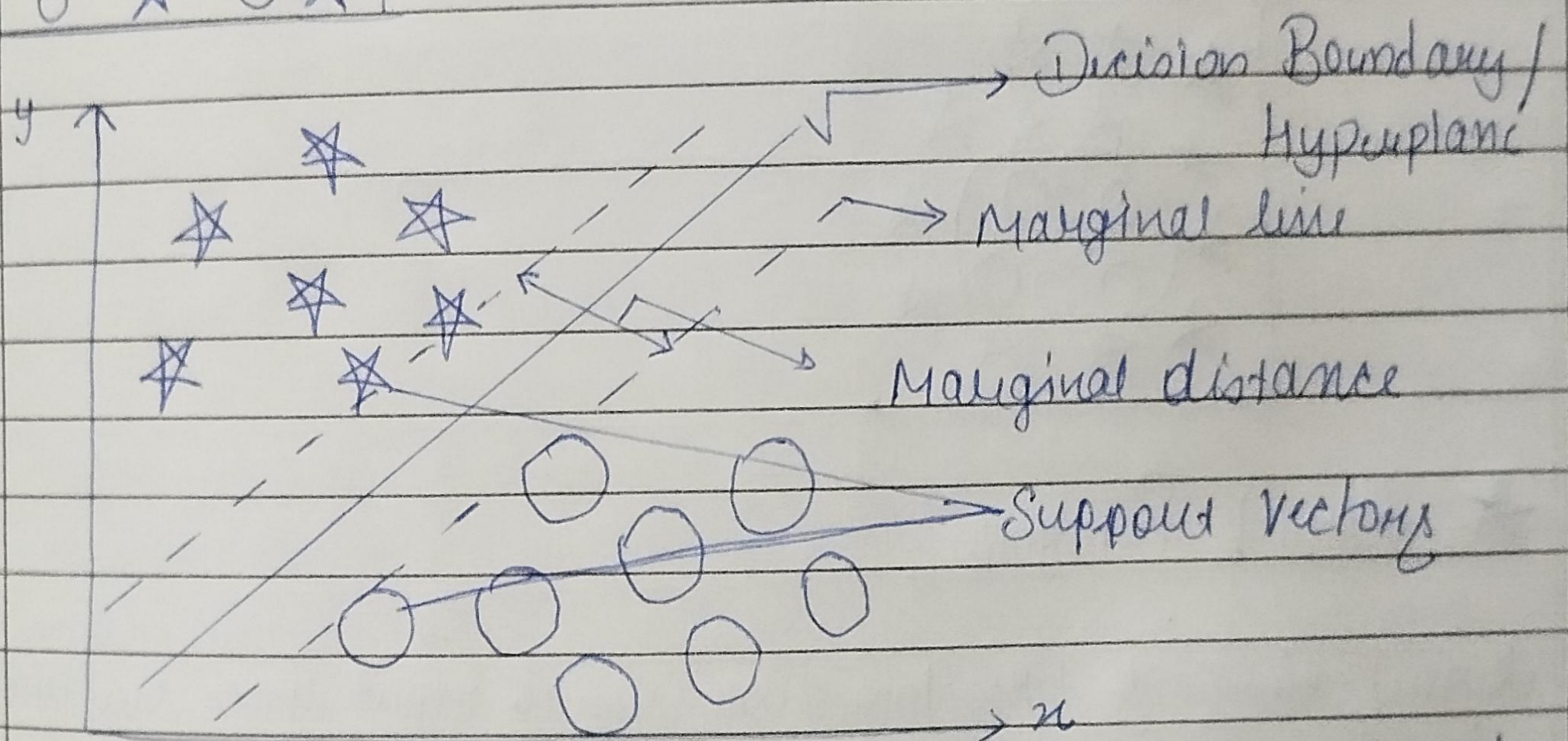
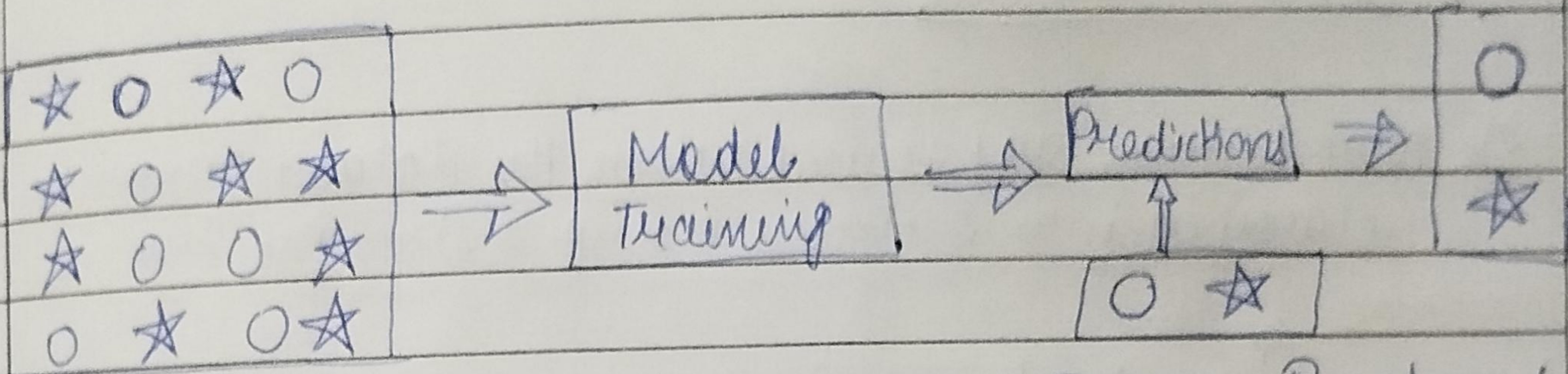
```
model.predict(x_test)
```

```
x_test
```

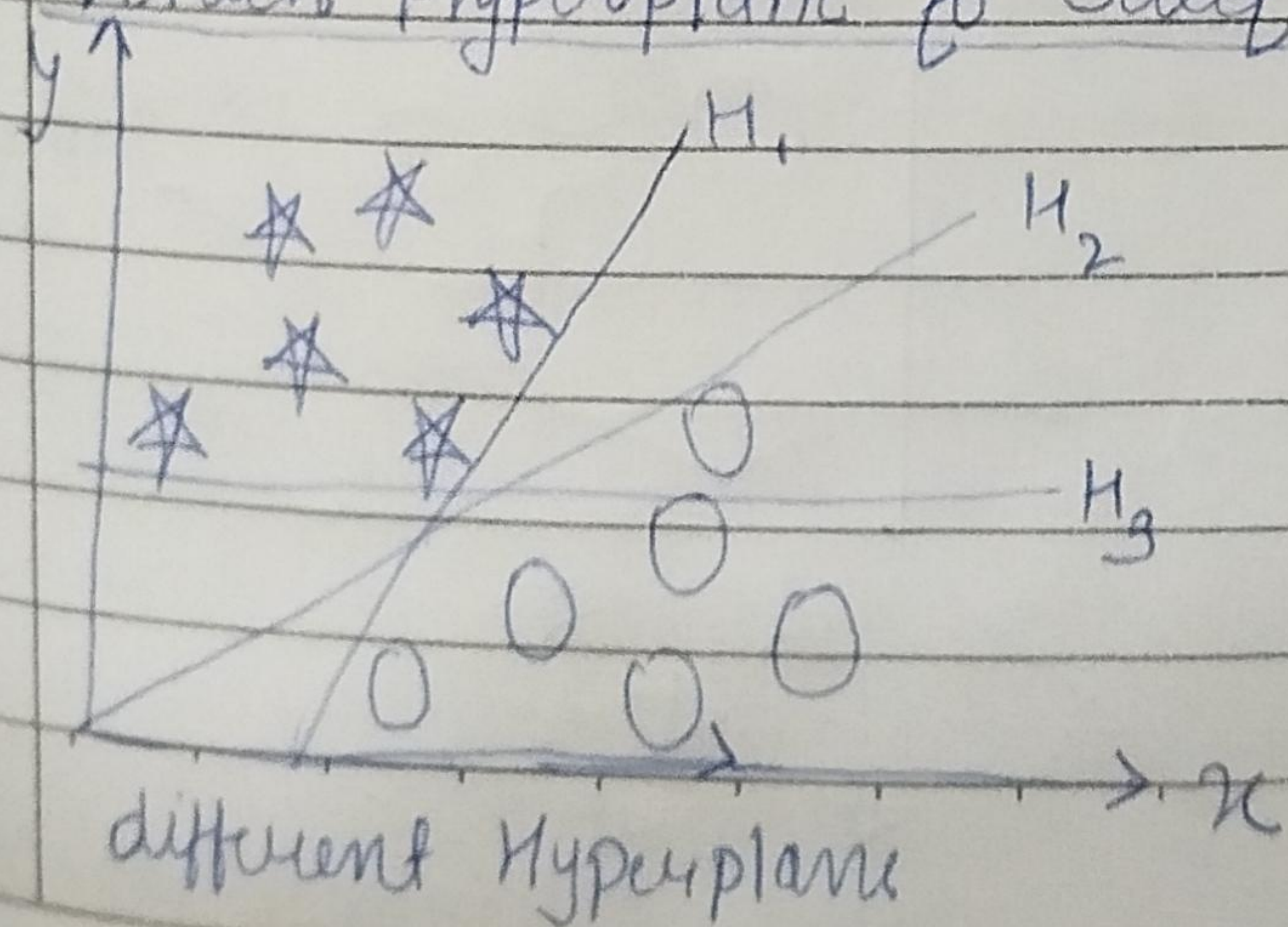
```
model.score(x_test, y_test)
```


★ Support Vector Machine Algorithm (SVM)

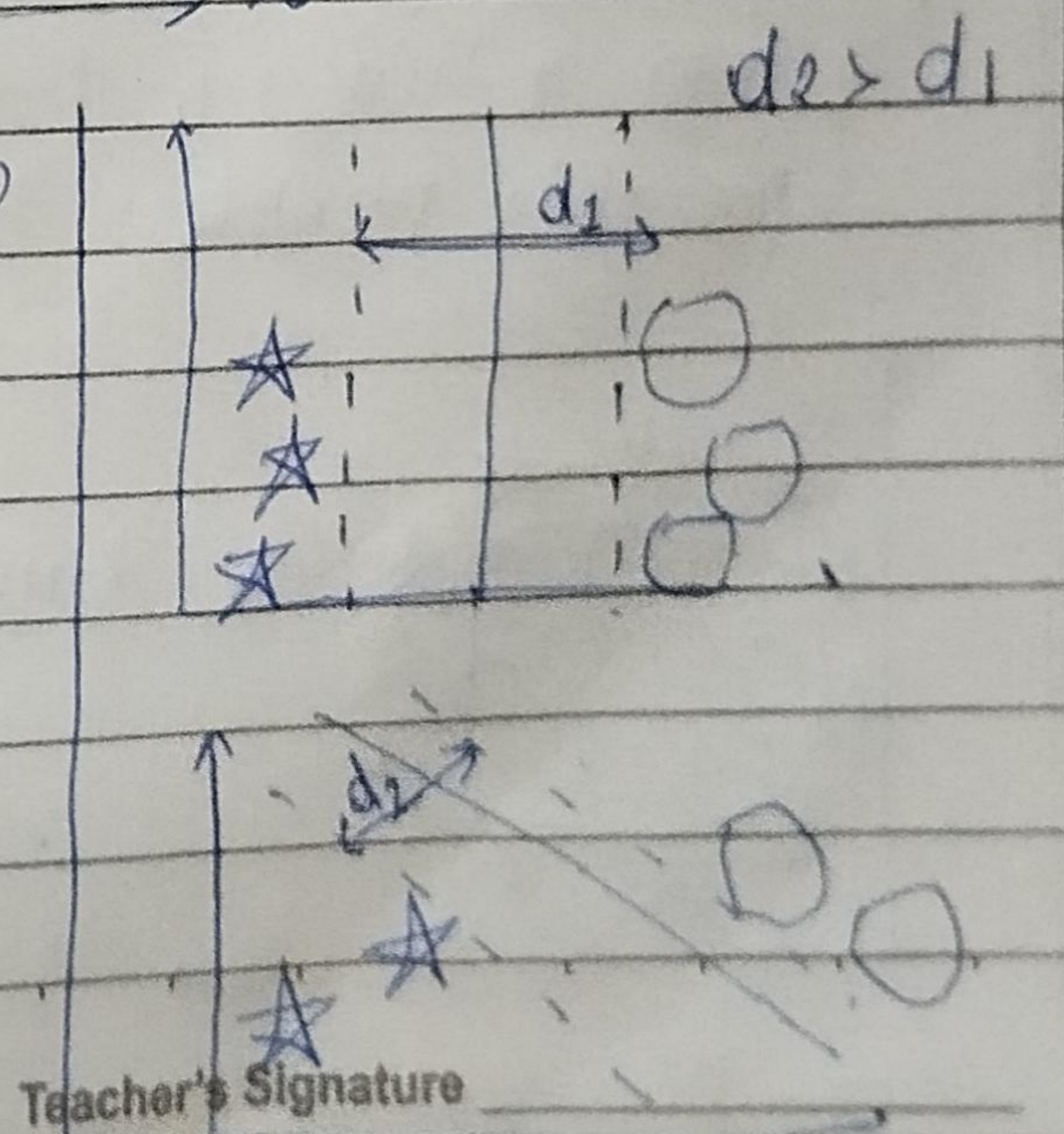
Support Vector Machine is a machine learning algorithm based on supervised learning. That can be used for both regression and classification problems.



★ Which Hyperplane to Select?



H_1, H_2, H_3

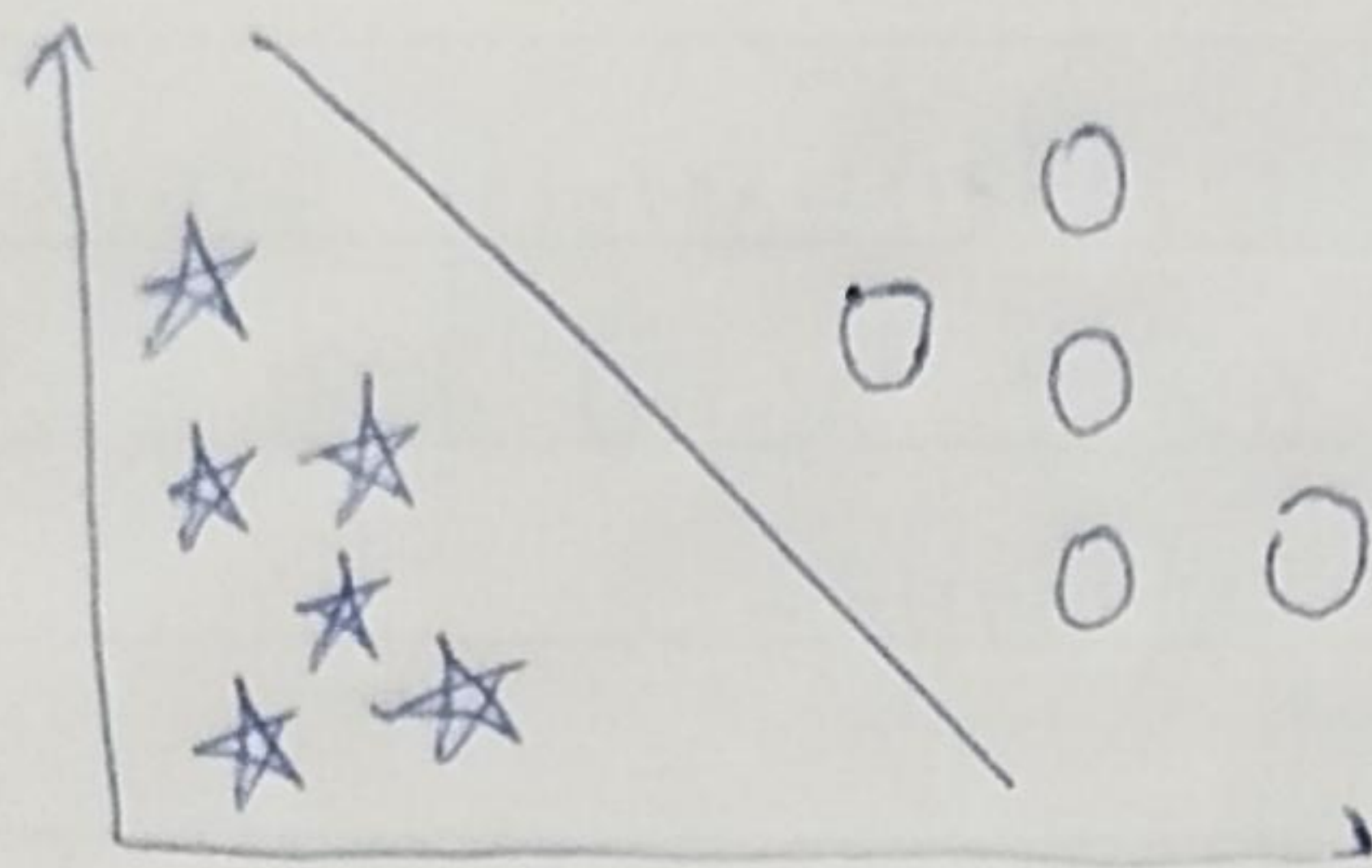


Teacher's Signature _____

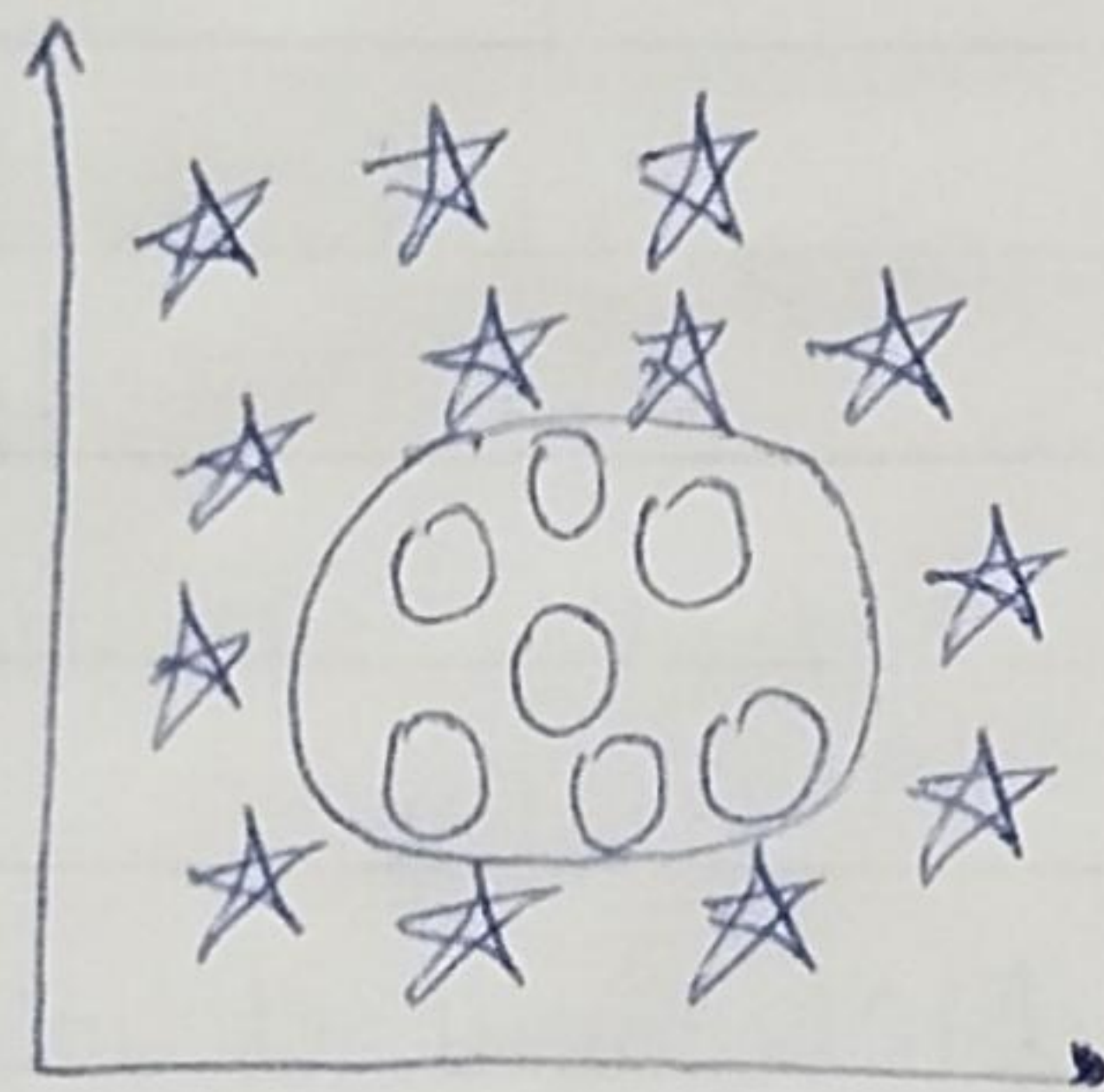
Maximum margin hyperplane

★ Types of SVM

1) Linear SVM is used when dataset can be classified into 2 classes using a straight line.

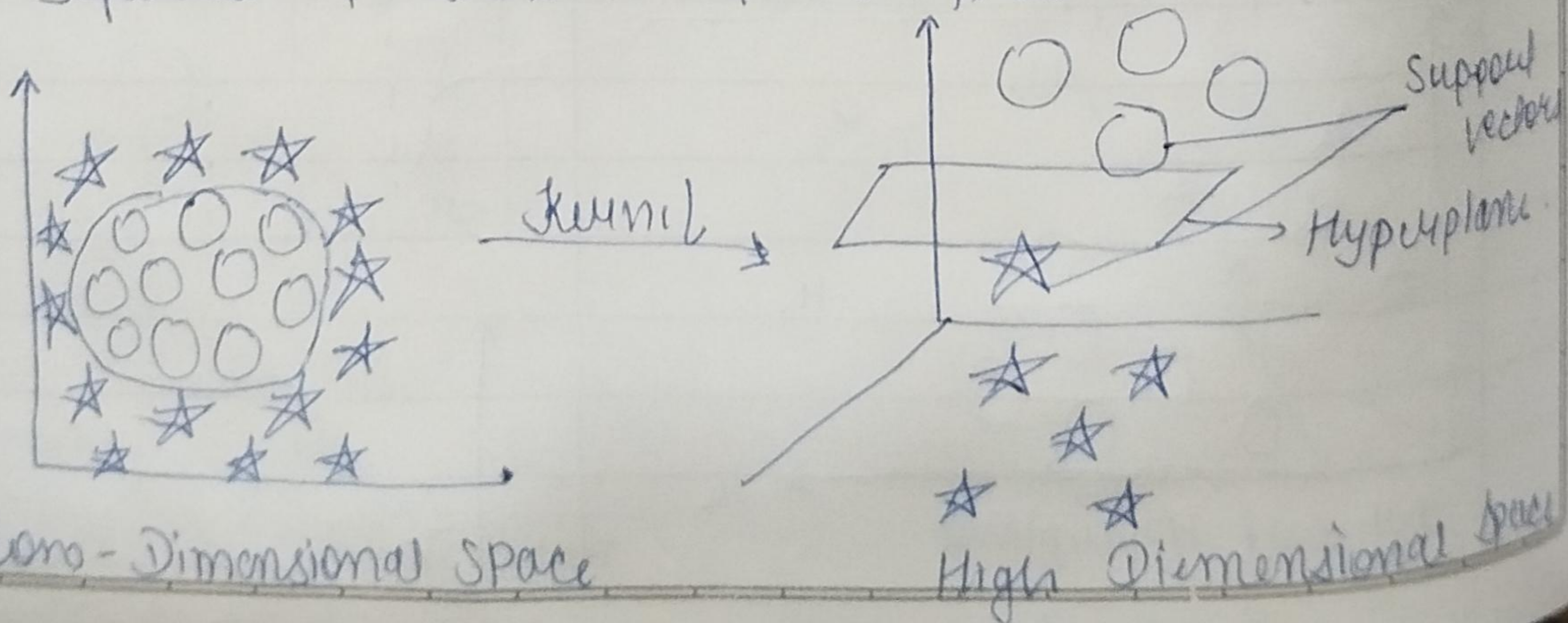


2) Non-linear SVM is used when the dataset cannot be classified into 2 classes using a straight line.



★ Kernel Function

Kernel Functions takes low dimensional input space and transform it into a higher-dimensional space, i.e., it converts not separable problem to separable problem.



Code

```
import pandas as pd  
data = pd.read_csv("C:\\Users\\Singh\\Downloads\\iris-dataset  
                .csv")
```

data

```
x = data.iloc[:, 0:4]
```

x

```
y = data.iloc[:, 5]
```

y

```
from sklearn.model_selection import train_test_split  
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2)
```

```
len(x_train)
```

```
len(x_test)
```

```
from sklearn.svm import SVC
```

```
model = SVC(kernel = 'linear')
```

```
model.fit(x_train, y_train)
```

```
model.predict(x_test)
```

```
x_test
```

```
model.score(x_test, y_test)
```